

Specification et Preuves de Programmes

Devoir 1

ZATON N'jie - M1 I2A

18/20

Exercice 1: Formaliser

"A si B" veut dire si B alors A : $B \Rightarrow A$

"A est condition nécessaire pour B" veut dire B ne peut être vrai qu'avec A est vrai : $B \Rightarrow A$

F "A sauf si B" veut dire si B est vrai alors A peut être faux : $\neg B \Rightarrow A$ ou $A \vee B$

"A seulement si B" veut dire si A alors B : $A \Rightarrow B$

"A est condition suffisante pour B" veut dire si A alors B : $A \Rightarrow B$

4/5 "A bien que B" veut dire A et B : $A \wedge B$

"Non seulement A, mais B aussi" veut dire A et B : $A \wedge B$

"A et pourtant B" veut dire A et B : $A \wedge B$

F "A à moins que B" veut dire A ou B : $\neg B \Rightarrow A$ ou $A \vee B$

"Ni A, ni B" veut dire : $\neg A \wedge \neg B$ équivalent à $\neg(A \vee B)$

Exercice 2: why3

On considère l'ensemble de formules propositionnelles suivant:

1. $p \vee q$
2. $\neg q \vee r$
3. $p \vee \neg r$
4. $\neg p \vee q$
5. $\neg p \vee \neg q$

On souhaite déterminer si cet ensemble de formules est contradictoire.
Pour cela, on utilise l'outil why3 afin de vérifier si ces formules impliquent false.

Theory for

predicte p

predicte q

predicte r

goal contradiction:

$(p \vee q) \rightarrow$
 $(\neg q \vee r) \rightarrow$



Scanned with
TapScanner

2

```

(p ∨ not r) →
(not p ∨ q) →
(not p ∨ not) →
false
end

```

Vn

La vérification dans l'interface why3 montre que le but est prouvé. Donc l'ensemble de formules est contradictoire.

Exercice 3 : Logique du premier ordre

- 1) Formule 1: $(\forall z, s(a, z)) \Rightarrow \exists u, s(u, b)$ Si $s(a, z)$ est vrai pour tout z , alors en choisissant $z = b$ on sait que $s(a, b)$ est vrai. Puisqu'il existe au moins un élément ($u = a$) tel que $s(u, b)$ est vrai, alors $\exists u, s(u, b)$ est vrai.

2

Formule 2: $(\forall z, s(a, z)) \Rightarrow s(a, b)$ c'est une application directe de la règle d'instanciation universelle. Si la propriété est vraie pour tout z , elle est nécessairement vraie pour la constante b .

2)

theory Ex3

type t

predicates s : t → t → bool

constant a : t

constant b : t

goal formula 1:

$(\text{forall } z : t. s(a, z)) \rightarrow \text{exists } u : t. s(u, b)$

goal formula 2:

$(\text{forall } z : t. s(a, z)) \rightarrow s(a, b)$

end

2

1

3) Ces deux buts sont validés.



Exercice 4 : type et théorie de formules propositionnelles en whyML

1) Définition du type $pf'a$

```
type pf'a =  
  | V 'a  
  | F  
  | Vrai  
  | Non (pf'a)  
  | Et (pf'a) (pf'a)  
  | Ou (pf'a) (pf'a)  
  | Imp (pf'a) (pf'a)  
  | Equiv (pf'a) (pf'a)
```

2

2) Définition des fonctions `taille`, `nb_at`, `hauteur` et `liste_at`

hors
sujet

```
use list.List  
use int.Int
```

```
let rec fonction taille (f : pf'a) : int =  
  match f with  
  | V _ | F | Vrai → 1  
  | Non f1 → 1 + taille f1  
  | Et f1 f2 | Ou f1 f2 | Imp f1 f2 | Equiv f1 f2 → 1 + taille f1 + taille f2  
end
```

0/2

```
let rec fonction nb_at (f : pf'a) : int =  
  match f with  
  | V _ → 1  
  | F | Vrai → 0  
  | Non f1 → nb_at f1  
  | Et f1 f2 | Ou f1 f2 | Imp f1 f2 | Equiv f1 f2 → nb_at f1 + nb_at f2  
end
```

Exercice 5 : Arbre binaires

1) Pour stocker les données un quantum dans les nœuds internes, nous définissons deux constructeurs :

- Un constructeur de base qui ne prend aucun argument et représente une terminaison d'arbre sans données. (Feuille)

- Un constructeur récursif qui prend trois arguments : un sous-arbre gauche, une donnée de type α et un sous-arbre droit (Nœud)

1



Ex-2) type arbre 'a =
 | Feuille
 | Noeud (arbre 'a) 'a (arbre 'a)

Exercice 6 : Propriétés de tableaux d'entiers.

Theory PropriétésTableaux
 use int.Int
 use array.Array

predicate nextSame (t: array int) =
 forall i. 0 <= i < length t - 1 → t[i] = t[i+1]

predicate croissant-decroissant (t: array int) (i: int) =
 0 <= i < length t ∧
 (forall j k. 0 <= j <= k <= i → t[j] <= t[k]) ∧
 (forall j k. i <= j <= k < length t → t[j] >= t[k])

predicate alternance-zero-un (t: array int) =
 let n = length t in
 (exist k. n = 2 * k) ∧
 (forall j. 0 <= 2 * j < n → t[2 * j] = 0) ∧
 (forall j. 0 <= 2 * j + 1 < n → t[2 * j + 1] = 1)

end

